

We can configure consistency on a Cassandra cluster, data centre, or as per individual read or write operations.

Tunable consistency is one of the strongest features of Cassandra. There are two types of consistency—strong and eventual. To ensure that data is written and read correctly, Cassandra extends the concept of eventual consistency by offering tunable consistency. Tunable data consistency allows individual read or write operations to be as strongly consistent as required by the client application. The consistency level of each read or write operation can be set, so that the data returned is more or less consistent, based on need.

**Data compression:** Data can be compressed up to 80 per cent without any performance overhead.

**Cassandra query language:** Cassandra provides a query language that is very similar to the SQL language. It helps developers moving from a relational database to Cassandra.

## Architecture of Apache Cassandra

Apache Cassandra's architecture offers the ability to scale, perform and provide continuous uptime. Rather than using a legacy master-slave or a manual and difficult-to-maintain sharded architecture, Apache Cassandra has a masterless 'ring' design that is elegant, easy to set up, and easy to maintain. It has a peer-to-peer distributed system across its nodes, and data is distributed among all the nodes in a cluster.

All nodes play an identical role—there is no concept of a master node, with all nodes communicating with each other equally.

In an Apache Cassandra cluster, each node is capable of handling large amounts of data and thousands of concurrent users or operations per second—even across multiple data centres. This is done with as much ease as when managing much smaller amounts of data and user traffic.

Apache Cassandra's architecture also ensures that, unlike other master-slave or sharded systems, it has no single point of failure and therefore is capable of offering true continuous availability and uptime—users need to simply add new nodes to an existing cluster without having to take it down.

## Key structures

The various structures of Apache Cassandra, along with a brief description of each, follows.

**Node:** This is where you store your data. It is the basic infrastructure component of Cassandra.

**Data centre:** This is a collection of related nodes. This could be a physical data centre or a virtual one. Different workloads should use separate data centres, either physical or virtual. Replication is set by the data centre. Using separate data centres prevents Cassandra transactions from being impacted by other workloads and keeps requests close to each other for lower latency. Depending on the replication factor, data can be written to multiple data centres. However, data centres should never span physical locations.



Figure 1: Apache Cassandra

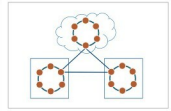


Figure 2: Cassandra supports multiple data centre and cloud deployments

**Cluster:** A cluster contains one or more data centres. It can span physical locations.

**Commit log:** All data is written first to the commit log for durability. After all its data has been flushed to SSTables, it can be archived, deleted or recycled.

**Table:** This is a collection of ordered columns fetched by rows. A row consists of columns and has a primary key. The first part of the key is a column name.

**SSTable:** This is a sorted string table (SSTable) and is an immutable data file to which Cassandra writes memtables periodically. SSTables are append-only, stored on disk sequentially and maintained for each Cassandra table.

## The Apache Cassandra data model

The data model of Cassandra is different from the relational DBMS. Cassandra does not support joins or sub-queries like in RDBMS. Instead, Cassandra emphasises denormalisation through features like collections.

Cassandra is basically a key-value and a column-oriented (or tabular) database. Rows are organised into tables—the first component of a table's primary key is the partition key. Within a partition, rows are clustered by the remaining columns of the key. Other columns can be indexed separately from the primary key.

The Cassandra data model consists of keyspace, column families, columns and rows.

**Keyspace:** This is the outermost container for your application data. It is similar to the schema in a relational database. The keyspace can include operational elements, such as the replication factor and data centre awareness. Keyspace is a group of many column families.

**Column family:** A column family is a container for an ordered collection of rows, each of which is itself an ordered collection of columns. A column family is similar to a table in an RDBMS and is a logical separation of similar data.

**Column:** This is a basic data structure of Cassandra with three values—name, value and timestamp.

**Super column:** The super column stores a map of sub-columns.

**Row:** This is a collection of columns labelled with a name.

## Different use cases of Apache Cassandra

Apache Cassandra can be used for various applications. Here are some use cases where Apache Cassandra is the best choice compared to other NoSQL databases.